

# DW\_lp\_piped\_fp\_sum3

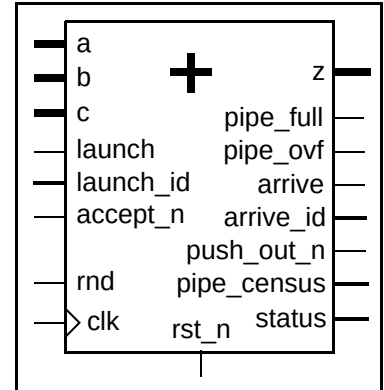
## Low Power Pipelined 3-input Floating Point Adder

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

### Features and Benefits

- The precision format is parameterizable for either IEEE single, double precision, or a user-defined custom format
- Hardware for denormal numbers of IEEE 754 standard is selectively provided.
- Fully compatible with the IEEE 754 standard with proper set of parameters
- DesignWare datapath generator is employed for better timing and area
- Saves power by only enabling stages as needed
- Saves power based on input data patterns
- Parameter controlled pipeline stages
- Flow control to interface directly to FIFO
- Flow control interfaces to another managed pipe
- Bubble removal extends depth of subsequent FIFO by pipe depth
- Parameter sized identifier tracks data operations

### Revision History



### Description

DW\_lp\_piped\_fp\_sum3 is a floating point operator that adds three floating point operands: a, and b, and c, to produce a floating point sum, z. The input rnd is a 3-bit rounding mode (see [Rounding Modes](#) in the *Datapath Floating-Point Overview*) and the output status is an 8-bit status flag. The operation is conditionally managed by the DW\_lp\_pipe\_mgr, a pipeline controller (if enabled when *no\_pm* = 0). The component is configured with a user selectable number of *stages* of logic, allowing design compiler to optimize the logic between register stages to reduce power and area. DC is able to take advantage of the enable signals provided in the design to optimize the combinational logic throughout the addition operation. When *no\_pm* = 0, the pipeline manager can manage the enabling of register stages of a pipeline based on *launch* requests that track the progress of a pipelined operation. Enabling stages of the pipeline only when they need to be clocked reduces dynamic power and is further enhanced through clock gate insertion (which requires Power Compiler). Launch requests can be accompanied by a *launch\_id* which can be used as a tag to identify operation results as they pass out of the pipe. The *pipe\_census* output monitors the number of operations in the pipe.

When *no\_pm* = 1, DW\_lp\_piped\_fp\_sum3 satisfies the need to speed up designs via pipelining and still enable register re-timing capability as well as providing identification handle used tracking each initiated transaction. However, low power and flow control can only be obtained by setting *no\_pm* = 0, thus using the

pipeline management mechanism (DW\_lp\_pipe\_mgr). For implementation specific details on the actual 3-input add function, see the [DW\\_fp\\_sum3](#) datasheet.

Component pins are described in [Table 1-1](#) and configuration parameters are described in [Table 1-2](#).

**Table 1-1 Pin Description**

| Pin Name    | Width                                                                                 | Direction | Function                                                                                                                                 |
|-------------|---------------------------------------------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------|
| a           | $(sig\_width + exp\_width + 1)$ bits                                                  | Input     | Input data                                                                                                                               |
| b           | $(sig\_width + exp\_width + 1)$ bits                                                  | Input     | Input data                                                                                                                               |
| c           | $(sig\_width + exp\_width + 1)$ bits                                                  | Input     | Input data                                                                                                                               |
| rnd         | 3 bits                                                                                | Input     | Rounding mode; supports all rounding modes described in the <a href="#">Datapath Floating-Point Overview</a>                             |
| z           | $(sig\_width + exp\_width + 1)$ bits                                                  | Output    | $(a + b) + c$                                                                                                                            |
| status      | 8 bits                                                                                | Output    | Status flags for the result z<br>For details, see <a href="#">STATUS Flags</a> in the <a href="#">Datapath Floating-Point Overview</a> . |
| launch      | 1 bit                                                                                 | Input     | Active high control input to launch data into pipe                                                                                       |
| launch_id   | $id\_width$ bits                                                                      | Input     | ID of launch                                                                                                                             |
| pipe_full   | 1 bit                                                                                 | Output    | Status flag indication no available slot in pipe                                                                                         |
| pipe_ovf    | 1 bit                                                                                 | Output    | Error flag indicating pipe overflow (data lost)                                                                                          |
| accept_n    | 1 bit                                                                                 | Input     | Flow control input (active low)                                                                                                          |
| arrive      | 1 bit                                                                                 | Output    | z result is valid                                                                                                                        |
| arrive_id   | $id\_width$ bits                                                                      | Output    | launch_id from the originating launch that produced the z result                                                                         |
| push_out_n  | 1 bit                                                                                 | Output    | Optional output used with FIFO (active low)                                                                                              |
| pipe_census | $\text{ceiling}(\log_2(\text{maximum}(1, in\_reg + (stages-1) + out\_reg) + 1))$ bits | Output    | Output bus indicating the number of pipe stages currently occupied                                                                       |

**Table 1-2 Parameter Description**

| Parameter | Values        | Description                                                             |
|-----------|---------------|-------------------------------------------------------------------------|
| sig_width | 2 to 253 bits | Word length of fraction field of floating point numbers a, b, c, and z  |
| exp_width | 3 to 31 bits  | Word length of biased exponent of floating point numbers a, b, c, and z |

**Table 1-2 Parameter Description (Continued)**

| Parameter       | Values                  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ieee_compliance | 0 or 1                  | <p>Level of support for IEEE 754:</p> <ul style="list-style-type: none"> <li>0: No support for IEEE 754 NaNs and denormals; NaNs are considered infinities and denormals are considered zeros</li> <li>1: Fully compliant with the IEEE 754 standard, including support for NaNs and denormals</li> </ul> <p>For more, see <a href="#">IEEE 754 Compatibility</a> in the <i>Datapath Floating-Point Overview</i>.</p>                                                                                                                                                                                                                          |
| arch_type       | 0 or 1<br>Default: 0    | <p>Controls the use of an alternative architecture</p> <p>0: Selects previous architecture</p> <p>1: Selects alternative architecture; when value is 1, <i>ieee_compliance</i> must be 0</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| op_iso_mode     | 0 to 4<br>Default: 0    | <p>Operand isolation mode (controls datapath gating for minPower flow)</p> <p>Allows you to set the style of minPower datapath gating for this module</p> <ul style="list-style-type: none"> <li>0: Use the DW_lp_op_iso_mode<sup>a</sup> synthesis variable</li> <li>1: 'none'</li> <li>2: 'and'</li> <li>3: 'or'</li> <li>4: Preferred gating style: 'and'</li> </ul> <p>Datapath gating is inserted only when there are no input registers on the operands at the component boundary. When inserted, datapath gating circuits are placed immediately after the input ports of the component (see <a href="#">Figure 1-3</a> on page 7).</p> |
| id_width        | 1 to 1024<br>Default: 1 | Width of input <i>launch_id</i> and output <i>arrive_id</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| in_reg          | 0 or 1<br>Default: 0    | <p>Input register control</p> <ul style="list-style-type: none"> <li>0: No input register</li> <li>1: Include input register</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| stages          | 1 to 1022<br>Default: 2 | Number of logic stages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| out_reg         | 0 or 1<br>Default: 0    | <p>Output register control</p> <ul style="list-style-type: none"> <li>0: No output register</li> <li>1: Include output register</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| no_pm           | 0 or 1<br>Default: 1    | <p>No pipeline management used</p> <ul style="list-style-type: none"> <li>0: Use pipeline management</li> <li>1: Do not use pipeline management</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| rst_mode        |                         | <p>Reset mode</p> <ul style="list-style-type: none"> <li>0: Asynchronous reset mode for <i>rst_n</i></li> <li>1: Synchronous reset mode for <i>rst_n</i></li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

- a. The DW\_lp\_op\_iso\_mode synthesis variable is available only in Design Compiler.  
DW\_lp\_op\_iso\_mode sets a global style of datapath gating. To use the global style, set *op\_iso\_mode* to '0'. Note that If the *op\_iso\_mode* parameter is set to '0' and DW\_lp\_op\_iso\_mode is either not set or set to 0', then no datapath gating is inserted for this component.

**Table 1-3 Synthesis Implementations**

| Implementation Name | Function        | License Feature Required                                                                                                                  |
|---------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| rtl                 | Synthesis model | <ul style="list-style-type: none"><li>■ DesignWare (P-2019.03 and later)</li><li>■ DesignWare-LP<sup>a</sup> (before P-2019.03)</li></ul> |

- a. For Design Compiler versions before P-2019.03, see [“Enabling minPower”](#) on page 9.

**Table 1-4 Simulation Models**

| Model                                   | Function                             |
|-----------------------------------------|--------------------------------------|
| DW03.DW_LP_PIPED_fp_sum3_CFG_SIM        | Design unit name for VHDL simulation |
| dw/dw03/src/DW_lp_piped_fp_sum3_sim.vhd | VHDL simulation model source code    |
| dw/sim_ver/DW_lp_piped_fp_sum3.v        | Verilog simulation model source code |

## Functional Description

### Block Diagram

Figure 1-1 shows the DW\_lp\_piped\_fp\_sum3 component with no pipeline management ( $no\_pm = 1$ ).

Figure 1-1 DW\_lp\_piped\_fp\_sum3 Block Diagram ( $no\_pm = 1$ )

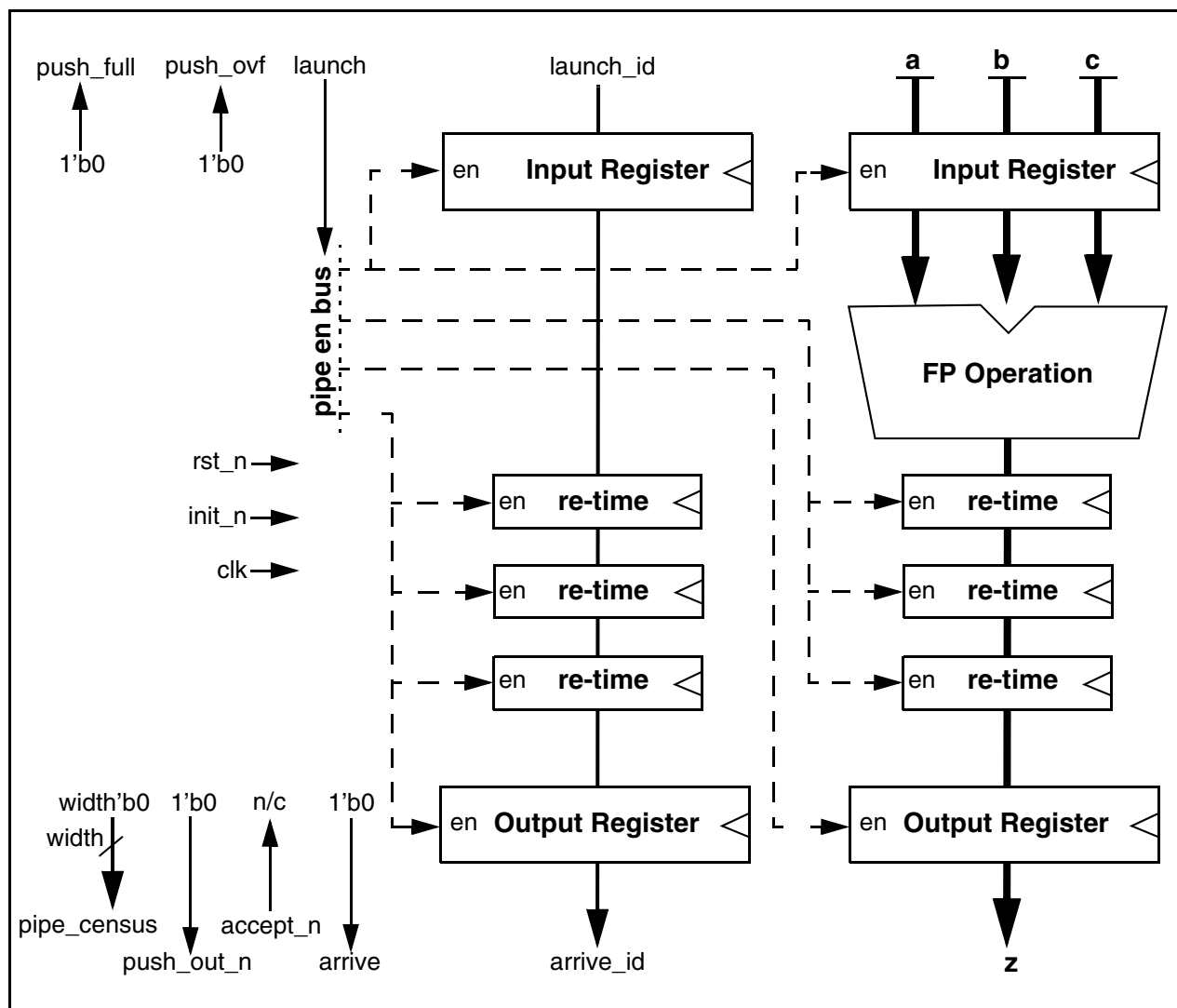
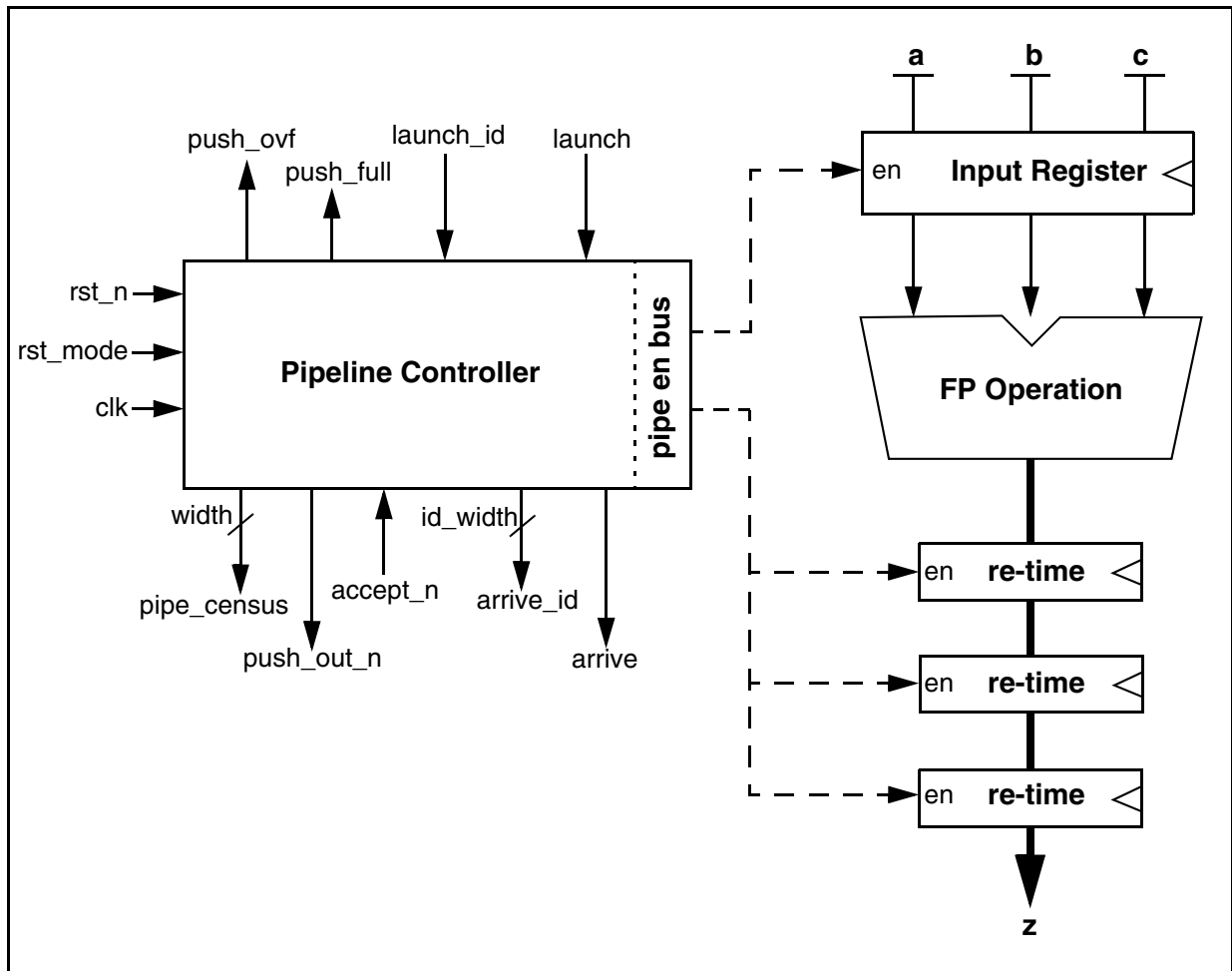


Figure 1-2 shows the DW\_lp\_piped\_fp\_sum3 component with pipeline management enabled ( $no\_pm = 0$ ):

Figure 1-2 DW\_lp\_piped\_fp\_sum3 Block Diagram ( $no\_pm = 0$ )



When no pipeline manager is implemented ( $no\_pm = 1$ ), the outputs are purely pipelined results initiated by  $a$ ,  $b$  and  $c$  when  $launch = 1$ . The pipeline is disabled (stalls) when  $launch = 0$ . Parameters  $in\_reg$ ,  $stages$ , and  $out\_reg$  define the number of register levels as shown below in Table 1-5. To assist in tracking each set of  $a$ ,  $b$  and  $c$  addition operations, transaction identifiers can be driven into  $launch\_id$  and monitored on the pipelined output  $arrive\_id$  to locate the associated results. That is, a unique  $launch\_id$  value per  $a$ ,  $b$ ,  $c$  and  $rnd$  will make its way through the pipeline and show up at  $arrive\_d$  along with its corresponding sum ( $z$ ), and status results.

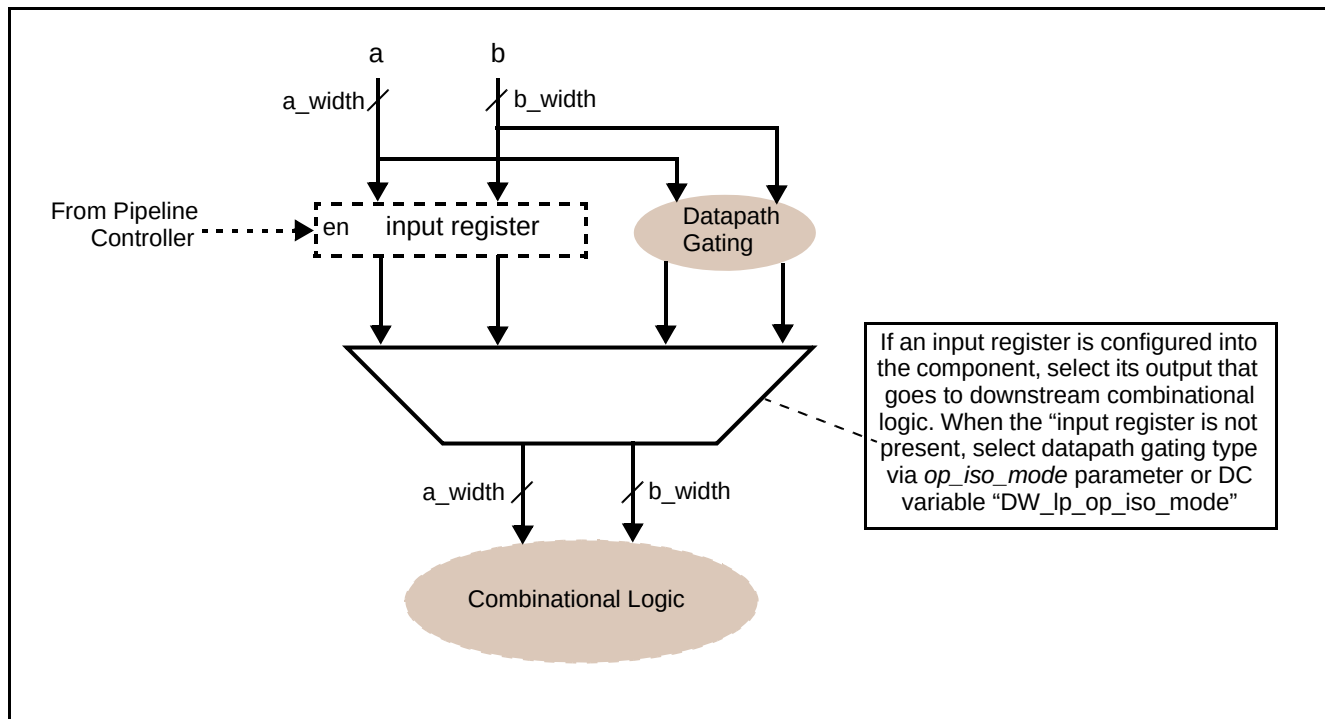
Table 1-5 Number of Pipeline Register Levels

| $in\_reg$ | $out\_reg$ | Number of Pipeline Register Levels |
|-----------|------------|------------------------------------|
| 0         | 0          | $stages - 1$                       |
| 0         | 1          | $stages$                           |
| 1         | 0          | $stages$                           |

**Table 1-5 Number of Pipeline Register Levels (Continued)**

| <i>in_reg</i> | <i>out_reg</i> | Number of Pipeline Register Levels |
|---------------|----------------|------------------------------------|
| 1             | 1              | <i>stages</i> + 1                  |

Figure 1-3 shows where datapath gating is inserted when the *op\_iso\_mode* parameter enables it.

**Figure 1-3 Location of Datapath Gating (If Inserted)**

## Pipelining

The DW\_lp\_piped\_fp\_sum3 is configurable embed pipeline register levels. Setting the value for the parameters *in\_reg*, *stages*, and *out\_reg* (see Table 1-5 on page 6) determines the number of pipeline register level(s) that are inserted. Therefore, depending on the parameter *in\_reg*, *stages*, and *out\_reg* settings, the number of clock cycles for the sum (*z*) and *status* results to propagate out varies.

This DW\_lp\_piped\_fp\_sum3 is designed to make it easy to pipeline floating point addition logic using the register retiming features of Design Compiler (DC). It also contains parameter controlled input and output registers which will stay in place at their respective block boundary, that is, they are not allowed to be moved by DC register retiming features. The input and output registers are not available when using DC versions earlier than C-2009.06.

The parameter *stages* refers to the number of logic stages desired after register retiming is performed. The number of register levels is not necessarily the same as the number of logic stages. If no input or output registers are used (*in\_reg* = 0 or *out\_reg* = 0), then there is one fewer register level than logic stages. If either an input register or output register is specified, then the number of register levels is the same as the number of logic stages. If both input and output registers are specified, then the number of register levels is the

number of logic *stages* + 1. Refer to [Table 1-5](#) on page 6 for a description. The number of pipeline register levels that can be retimed is always *stages* - 1.

## Pipeline Control and Power Savings

When pipeline management is enabled (*no\_pm* = 0), running in parallel to the pipeline register levels is pipeline control logic (as seen in [Figure 1-2](#) on page 6) that monitors the activity. In cases where there is inactivity on a particular register level of the pipeline, the pipeline control disables those levels to promote power savings. Furthermore, if using the Synopsys Power Compiler tool, the presence of the pipeline control and its wiring to the pipeline register levels provides an opportunity for increased power reduction in the form of clock gating.

Along with the potential power savings that the pipeline control provides, it can be utilized to improve performance in cases where intermittent launch operations are present and there contains first-in first-out (FIFO) structures upstream and downstream of the DW\_lp\_piped\_fp\_sum3. The handshake is made between the DW\_lp\_piped\_fp\_sum3 and the external FIFOs via the *accept\_n* and *pipe\_full* ports. Effectively, the DW\_lp\_piped\_fp\_sum3 can be considered part of the external FIFO structures. The performance gain comes when inactive (bubbles) stages are detected. These pipeline 'bubbles' are removed to produce a contiguous set of active pipeline stages. The result is empty pipeline slots at the head of (or entering) the DW\_lp\_piped\_fp\_sum3 pipeline for new operations to be launched. Advancing the shifting of operations through the pipeline when a valid product result is available (*arrive* = 1) is controlled by the *accept\_n* input. When the multiplier pipeline is full of active entries, the *pipe\_full* output is 1. To disable this feature in cases where no external FIFOs are present, set the *accept\_n* input to 0 which will effectively eliminate any flow control. At the same time, the *pipe\_full* output would always be 0.

To assist in tracking of 'launched' operands, the pipeline control logic provides interface ports called *launch\_id* and *arrive\_id*. The *launch\_id* input is assigned a value during an active launch operation. Given that *launch\_id* values are unique in successive launch operations, the product results can be distinguished from one another with the assertion of *arrive* and the associated *arrive\_id*. The *arrive\_id* is the *launch\_id* from the originating launch that produced the valid product result.

When *no\_pm* = 1 the pipeline manager is not used, and the *launch* input is connected directly to the enable line of the pipeline registers. The pipeline stalls when *launch* = 0.

## No Pipeline Register Levels Specified

In cases where no pipelining is required (*no\_pm* = 1) through the DW\_lp\_piped\_fp\_sum3 (*in\_reg* = 0, *stages* = 1, and *out\_reg* = 0), the pipeline control flow control handshaking/status signals still remain active and meaningful with one exception. The *pipe\_census*, which is intended to count the number of active pipeline register levels, becomes irrelevant and is fixed to 0.

## Reset

### System Resets (synchronous or asynchronous)

Two system reset modes are available from the *rst\_n* input: asynchronous or synchronous. Asynchronous system reset is implemented when *rst\_mode* is 0 and synchronous system reset is applied when *rst\_mode* is 1.

During reset conditions, all the output ports are set to '0'.

## Suppressing Warning Messages During Verilog Simulation

The Verilog simulation model includes macros that allow you to suppress warning messages during simulation.

To suppress all warning messages for all DWBB components, define the DW\_SUPPRESS\_WARN macro in either of the following ways:

- Specify the Verilog preprocessing macro in Verilog code:  

```
`define DW_SUPPRESS_WARN
```
- Or, include a command line option to the simulator, such as:  

```
+define+DW_SUPPRESS_WARN
```

 (which is used for the Synopsys VCS simulator)

The warning messages for this model include the following:

- If values other than 1 or 0 are present on a clock port, the following message is displayed:

```
WARNING: <instance_path>.<clock_name>_monitor:  
at time = <timestamp>, Detected unknown value, x, on <clock_name> input.
```

To suppress only this warning message for all DWBB components, use the following macro:

- Define the DW\_DISABLE\_CLK\_MONITOR macro. You can define this macro in the following ways:
  - Specify the Verilog preprocessing macro in Verilog code:  

```
`define DW_DISABLE_CLK_MONITOR
```
  - Or, include a command line option to the simulator, such as:  

```
+define+DW_DISABLE_CLK_MONITOR
```

 (which is used for the Synopsys VCS simulator)

This message is also suppressed using the DW\_SUPPRESS\_WARN macro explained earlier.

- If an invalid rounding mode has been detected on rnd, the following message is displayed:

```
WARNING: <instance_path>:  
at time = <timestamp>: Illegal rounding mode.
```

To suppress this message, use the DW\_SUPPRESS\_WARN macro explained earlier.

## Enabling minPower

In Design Compiler (version P-2019.03 and later) and Fusion Compiler, you can instantiate this component and use all its features without special settings.

For versions of Design Compiler before P-2019.03, enable minPower as follows:

```
set synthetic_library {dw_foundation.sldb dw_minpower.sldb}  
set link_library {* $target_library $synthetic_library}
```

## Related Topics

- [Datapath Floating-Point Overview](#)
- [DesignWare Building Blocks User Guide](#)

## HDL Usage Through Component Instantiation - VHDL

```
library IEEE,DWARE;
use IEEE.std_logic_1164.all;
use DWARE.DWpackages.all;
use DWARE.DW_Foundation_comp.all;

entity DW_lp_piped_fp_sum3_inst is
    generic (
        inst_sig_width : POSITIVE := 23;
        inst_exp_width  : POSITIVE := 8;
        inst_ieee_compliance : INTEGER := 0;
        inst_arch_type   : INTEGER := 0;
        inst_op_iso_mode : NATURAL := 0;
        inst_id_width    : POSITIVE := 8;
        inst_in_reg      : NATURAL := 0;
        inst_stages      : POSITIVE := 4;
        inst_out_reg     : NATURAL := 0;
        inst_no_pm       : NATURAL := 1;
        inst_rst_mode    : NATURAL := 0
    );
    port (
        inst_clk : in std_logic;
        inst_rst_n : in std_logic;
        inst_a : in std_logic_vector(inst_sig_width+inst_exp_width downto 0);
        inst_b : in std_logic_vector(inst_sig_width+inst_exp_width downto 0);
        inst_c : in std_logic_vector(inst_sig_width+inst_exp_width downto 0);
        inst_rnd : in std_logic_vector(2 downto 0);
        z_inst : out std_logic_vector(inst_sig_width+inst_exp_width downto 0);
        status_inst : out std_logic_vector(7 downto 0);
        inst_launch : in std_logic;
        inst_launch_id : in std_logic_vector(inst_id_width-1 downto 0);
        pipe_full_inst : out std_logic;
        pipe_ovf_inst : out std_logic;
        inst_accept_n : in std_logic;
        arrive_inst : out std_logic;
        arrive_id_inst : out std_logic_vector(inst_id_width-1 downto 0);
        push_out_n_inst : out std_logic;
        pipe_census_inst : out
        std_logic_vector(bit_width(maximum(1,inst_in_reg+(inst_stages-1)+inst_out_reg)+1)-1
        downto 0)
    );
end DW_lp_piped_fp_sum3_inst;

architecture inst of DW_lp_piped_fp_sum3_inst is

begin
```

```
-- Instance of DW_lp_piped_fp_sum3
U1 : DW_lp_piped_fp_sum3
    generic map ( sig_width => inst_sig_width, exp_width => inst_exp_width,
ieee_compliance => inst_ieee_compliance, arch_type => inst_arch_type, op_iso_mode =>
inst_op_iso_mode, id_width => inst_id_width, in_reg => inst_in_reg, stages =>
inst_stages, out_reg => inst_out_reg, no_pm => inst_no_pm, rst_mode => inst_rst_mode )
    port map ( clk => inst_clk, rst_n => inst_rst_n, a => inst_a, b => inst_b, c =>
inst_c, rnd => inst_rnd, z => z_inst, status => status_inst, launch => inst_launch,
launch_id => inst_launch_id, pipe_full => pipe_full_inst, pipe_ovf => pipe_ovf_inst,
accept_n => inst_accept_n, arrive => arrive_inst, arrive_id => arrive_id_inst,
push_out_n => push_out_n_inst, pipe_census => pipe_census_inst );

end inst;
```

## HDL Usage Through Component Instantiation - Verilog

```
module DW_lp_piped_fp_sum3_inst( inst_clk, inst_rst_n, inst_a, inst_b, inst_c,
                                inst_rnd, z_inst, status_inst, inst_launch, inst_launch_id,
                                pipe_full_inst, pipe_ovf_inst, inst_accept_n, arrive_inst, arrive_id_inst,
                                push_out_n_inst, pipe_census_inst );

parameter sig_width = 23;
parameter exp_width = 8;
parameter ieee_compliance = 0;
parameter arch_type = 0;
parameter op_iso_mode = 0;
parameter id_width = 8;
parameter in_reg = 0;
parameter stages = 4;
parameter out_reg = 0;
parameter no_pm = 1;
parameter rst_mode = 0;

`define bit_width_MX_1__in_reg_P_stages_M_1_P_out_reg_P_1 2

input inst_clk;
input inst_rst_n;
input [sig_width+exp_width : 0] inst_a;
input [sig_width+exp_width : 0] inst_b;
input [sig_width+exp_width : 0] inst_c;
input [2 : 0] inst_rnd;
output [sig_width+exp_width : 0] z_inst;
output [7 : 0] status_inst;
input inst_launch;
input [id_width-1 : 0] inst_launch_id;
output pipe_full_inst;
output pipe_ovf_inst;
input inst_accept_n;
output arrive_inst;
output [id_width-1 : 0] arrive_id_inst;
output push_out_n_inst;
output [(`bit_width_MX_1__in_reg_P_stages_M_1_P_out_reg_P_1)-1 : 0] pipe_census_inst;

// Instance of DW_lp_piped_fp_sum3
DW_lp_piped_fp_sum3 #(sig_width, exp_width, ieee_compliance, arch_type,
op_iso_mode, id_width, in_reg, stages, out_reg, no_pm, rst_mode)
U1 ( .clk(inst_clk), .rst_n(inst_rst_n), .a(inst_a), .b(inst_b), .c(inst_c),
.rnd(inst_rnd), .z(z_inst), .status(status_inst), .launch(inst_launch),
.launch_id(inst_launch_id), .pipe_full(pipe_full_inst), .pipe_ovf(pipe_ovf_inst),
.accept_n(inst_accept_n), .arrive(arrive_inst), .arrive_id(arrive_id_inst),
.push_out_n(push_out_n_inst), .pipe_census(pipe_census_inst) );
```

endmodule

## Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet with visible change bars, click [here](#).

| Date           | Release       | Updates                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| September 2024 | DWBB_202409.0 | <ul style="list-style-type: none"> <li>Updated date and version; no technical changes</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| December 2023  | DWBB_202312.0 | <ul style="list-style-type: none"> <li>Updated date and version; no technical changes</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| July 2023      | DWBB_202212.5 | <ul style="list-style-type: none"> <li>Updated version and date</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| September 2021 | DWBB_202106.2 | <ul style="list-style-type: none"> <li>Adjusted the position of the <i>id_width</i> parameter in <a href="#">Table 1-2</a> on page 2 to match parameter order in RTL</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                 |
| July 2020      | DWBB_201912.5 | <ul style="list-style-type: none"> <li>Adjusted the description of the <i>ieee_compliance</i> parameter in <a href="#">Table 1-2</a> on page 2</li> <li>Adjusted content and title of “<a href="#">Suppressing Warning Messages During Verilog Simulation</a>” on page 9 and added the DW_SUPPRESS_WARN macro and the illegal round mode message</li> </ul>                                                                                                                                                                                                     |
| October 2019   | DWBB_201903.5 | <ul style="list-style-type: none"> <li>Added “Disabling Clock Monitor Messages”</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| March 2019     | DWBB_201903.0 | <ul style="list-style-type: none"> <li>Clarified the op_iso_mode parameter in <a href="#">Table 1-2</a> on page 2</li> <li>Clarified licensing requirements in <a href="#">Table 1-3</a> on page 4</li> <li>Clarified the figure titles for <a href="#">Figure 1-1</a> on page 5 and <a href="#">Figure 1-2</a> on page 6</li> <li>Added <a href="#">Figure 1-3</a> on page 7 to clarify datapath gating</li> <li>Added “<a href="#">Enabling minPower</a>” on page 9</li> <li>Added this Revision History table and the document links on this page</li> </ul> |

## Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

### Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

### Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

### Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

### Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

### Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.  
[www.synopsys.com](http://www.synopsys.com)

